

Software Design Document

Visually Essential Social (VES) — Blood Bank Donor Management System (BBDMS)



Submitted by:

Muhammad Adil Nadeem

Roll No: S25BINFT7M04028

Submitted to:

Prof. Dr. Musarat Karim

**Project Supervisor, Department of Information
Technology**

Department of Information Technology

Faculty of Computing

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

**Final Year Project — BS Information Technology, 7th Semester | June
2026**

Summary

This Software Design Document (SDD) provides the complete architectural and design blueprint for Visually Essential Social (VES) — a web-based social platform focused on Blood Bank Donor Management (BBDMS). It translates all functional and non-functional requirements into concrete design decisions, covering system architecture, module design, database schema, API structure, UI layout, and security controls.

Table of Contents

1. Introduction

- 1.1 Purpose of this Document
- 1.2 Project Overview
- 1.3 Scope
- 1.4 Intended Audience
- 1.5 Definitions and Abbreviations

2. System Overview

- 2.1 Problem Statement
- 2.2 Proposed Solution
- 2.3 User Roles

3. System Architecture

- 3.1 Architectural Overview
- 3.2 Technology Stack

4. Module Decomposition

- 4.1 Module Overview
- 4.2 Module Descriptions

5. Database Design

- 5.1 Entity-Relationship Summary
- 5.2 Detailed Table Schemas
- 5.3 Sample MySQL CREATE Statements

6. Backend / PHP Design

- 6.1 PHP File Structure and Routing
- 6.2 Session and Authentication Flow
- 6.3 Database Connection

7. User Interface Design

- 7.1 Design Principles
- 7.2 Page Inventory
- 7.3 Navigation Structure

8. Security Design

- 8.1 Authentication and Session Security
- 8.2 Security Controls Table

9. Use Case and Sequence Design

- 9.1 Sequence: Donor Registration
- 9.2 Sequence: Search Donor
- 9.3 Sequence: Admin Deletes Donor

10. Non-Functional Design Decisions

11. Project Folder Structure

12. Design Rationale

13. References

1. Introduction

1.1 Purpose of this Document

This Software Design Document (SDD) provides the complete architectural and design blueprint for Visually Essential Social (VES) — a web-based social platform focused on Blood Bank Donor Management (BBDMS). It translates all functional and non-functional requirements into concrete design decisions, covering system architecture, module design, database schema, API structure, UI layout, and security controls.

1.2 Project Overview

Visually Essential Social (VES) is a Final Year Project developed at the Islamia University of Bahawalpur. The platform serves a critical social need in Pakistan: connecting blood donors with recipients efficiently through a dedicated web-based system. The project is implemented as BBDMS — Blood Bank Donor Management System — a PHP and MySQL based web application that allows users to register as donors, search for donors by blood group and location, and allows administrators to manage the entire donor database.

Pakistan faces a chronic shortage of safe blood supply. VES addresses this by providing a centralized, easily accessible platform where verified blood donors can register their profiles and where hospitals, patients, and families can quickly find matching donors — potentially saving lives.

1.3 Scope

This SDD covers the following design areas:

System architecture and deployment model

Module decomposition and responsibilities

Complete MySQL database schema

PHP backend and routing design

User interface page structure

Security and authentication design

Non-functional design decisions

1.4 Intended Audience

Muhammad Adil Nadeem (Developer)

Prof. Dr. Musarat Karim (Project Supervisor)

Academic evaluators — Department of IT, IUB

1.5 Definitions and Abbreviations

Term / Abbreviation	Definition
VES	Visually Essential Social — the project name
BBDMS	Blood Bank Donor Management System — the core system
SDD	Software Design Document (this document)
SRS	Software Requirements Specifications
PHP	Hypertext Preprocessor — server-side scripting language
MySQL	Relational database management system
CRUD	Create, Read, Update, Delete — standard data operations
UI	User Interface
SQL	Structured Query Language — used to manage MySQL data
Admin	System administrator with full platform access
Donor	A registered user willing to donate blood
IUB	The Islamia University of Bahawalpur

2. System Overview

2.1 Problem Statement

In Pakistan, blood shortages are a persistent and life-threatening problem. When a patient urgently needs blood, their family is forced to search manually through contacts, social media posts, and hospital lists — wasting precious time. There is no centralized, reliable, and easily searchable database of willing blood donors available to the public.

2.2 Proposed Solution

VES (BBDMS) provides a structured, web-based platform where:

Blood donors can register their profiles with blood group, location, and contact information

Anyone in need can search the donor list by blood group or location instantly

Administrators can verify, manage, and moderate all donor records

The system maintains a clean, up-to-date, and searchable donor database

2.3 User Roles

User Role	Access Level	Responsibilities
Guest / Visitor	Public (no login)	View home page, search donors by blood group, view donor list, contact us
Registered Donor	Authenticated	Register profile, update own details, manage donation availability status
Administrator	Full Access	Manage all donors, add/edit/delete records, view system statistics, moderate content

Table 2.1 — VES User Roles

3. System Architecture

3.1 Architectural Overview

VES follows a classic Three-Tier Web Architecture. The user interacts with HTML/CSS pages rendered by PHP, which communicates with a MySQL database on the server. All three layers run on a local or hosted Apache/PHP server (XAMPP for development, or a shared hosting provider for production).

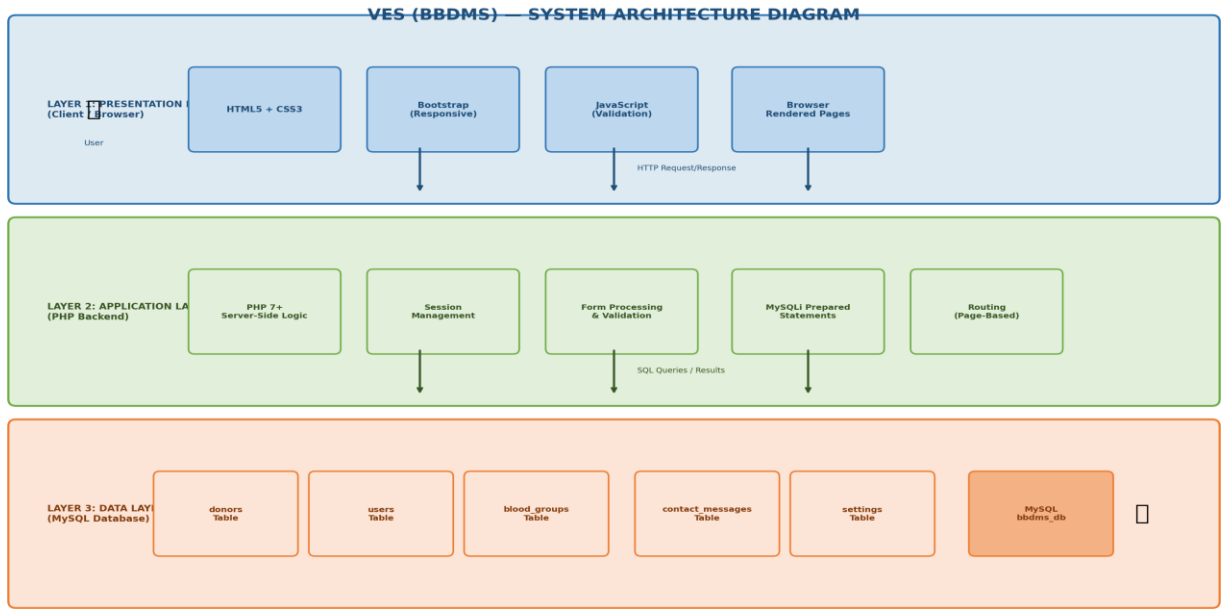


Figure 3.1 — VES Three-Tier Architecture Diagram

3.2 Technology Stack

Category	Technology	Purpose
Frontend	HTML5, CSS3, Bootstrap	Responsive user interface and page structure
Scripting	JavaScript	Client-side form validation and UI interaction
Backend	PHP 7+	Server-side logic, form processing, session auth
Database	MySQL	Relational data storage for all platform data
Web Server	Apache (XAMPP)	Local development server
Version Control	Git / GitHub	Source code management
Development IDE	VS Code / Sublime Text	Primary development environment

Table 3.1 — VES Technology Stack

4. Module Decomposition

The VES (BBDMS) system is divided into five core modules, each handling a specific domain of functionality.

4.1 Module Overview

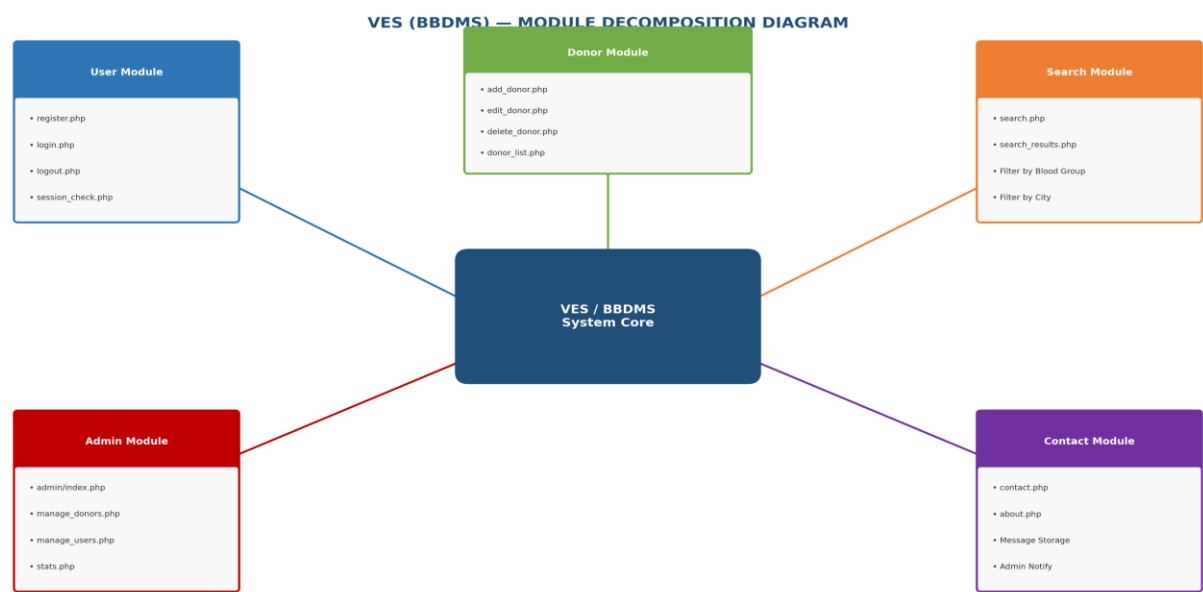


Figure 4.1 — VES Module Decomposition Diagram

4.2 Module Descriptions

4.2.1 User Module

Handles all user authentication, session management, and account-related operations for both donors and administrators.

Attribute	Detail
PHP Files	register.php, login.php, logout.php, session_check.php

Attribute	Detail
Key Operations	INSERT user, SELECT + verify password, START SESSION, DESTROY SESSION
Dependencies	MySQL users table, PHP sessions

4.2.2 Donor Module

Core module of the system — manages the complete lifecycle of donor records including registration, editing, and deletion.

Attribute	Detail
PHP Files	add_donor.php, edit_donor.php, delete_donor.php, donor_list.php, donor_detail.php
Key Operations	INSERT donor, UPDATE donor, DELETE donor, SELECT all donors, SELECT donor by ID
Dependencies	MySQL donors table, blood_groups table, User Module (auth check)

4.2.3 Search Module

Provides the critical public-facing functionality — searching for available donors by blood group or location without requiring login.

Attribute	Detail
PHP Files	search.php, search_results.php
Key Operations	SELECT donors WHERE blood_group = ? OR city = ? (parameterized queries)
Dependencies	MySQL donors table, blood_groups table

4.2.4 Admin Module

Provides privileged access for system administrators to manage all donor records, view platform statistics, and handle user accounts.

Attribute	Detail
PHP Files	admin/index.php, admin/manage_donors.php, admin/manage_users.php, admin/stats.php
Key Operations	Full CRUD on all tables; SELECT COUNT statistics; session-based admin auth
Dependencies	All database tables; Admin session validation middleware

4.2.5 Contact Module

Handles the public contact form, stores messages in the database, and provides the About Us information page.

Attribute	Detail
PHP Files	contact.php, about.php
Key Operations	INSERT contact_messages, SELECT messages in admin panel
Dependencies	MySQL contact_messages table

5. Database Design

VES uses MySQL as its relational database. The database name is bbdms_db. The schema consists of five tables designed to store all platform data including donors, users, blood groups, contact messages, and system settings.

5.1 Entity-Relationship Summary

Table Name	Primary Key	Relationships / Purpose
donors	donor_id (PK)	Core table — stores all donor profiles including blood group, city, contact
users	user_id (PK)	Stores registered user and admin accounts with hashed passwords
blood_groups	bg_id (PK)	Lookup table — lists all valid blood groups (A+, A-, B+, B-, O+, O-, AB+, AB-)
contact_messages	msg_id (PK)	Stores messages submitted via the public Contact Us form
settings	setting_id (PK)	Stores system-wide configuration values (site name, contact email, etc.)

Table 5.1 — VES Database Tables Summary

5.2 Detailed Table Schemas

5.2.1 donors Table

Column	Data Type	Constraint	Description
donor_id	INT	PK, AUTO_INCREMENT	Unique donor record ID
full_name	VARCHAR(100)	NOT NULL	Full name of the donor
blood_group	VARCHAR(5)	NOT NULL	Blood group (e.g., A+, O-, AB+)
age	INT	NOT NULL	Age of the donor (must be 18-60)
gender	ENUM	NOT NULL	Male / Female / Other

Column	Data Type	Constraint	Description
city	VARCHAR(100)	NOT NULL	City or district of the donor
address	TEXT	NULLABLE	Full address for location reference
phone	VARCHAR(20)	NOT NULL	Primary contact number (Pakistani format)
email	VARCHAR(150)	NULLABLE	Optional email address
is_available	TINYINT(1)	DEFAULT 1	1 = available to donate, 0 = not available
last_donation	DATE	NULLABLE	Date of last blood donation
registered_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last record update timestamp

5.2.2 users Table

Column	Data Type	Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user ID
full_name	VARCHAR(100)	NOT NULL	Full name of the user or admin
email	VARCHAR(150)	UNIQUE NOT NULL	Login email address
password	VARCHAR(255)	NOT NULL	bcrypt hashed password — never stored as plain text
role	ENUM	DEFAULT 'user'	user / admin — determines access level

Column	Data Type	Constraint	Description
is_active	TINYINT(1)	DEFAULT 1	1 = active account, 0 = suspended
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

5.2.3 blood_groups Table

Column	Data Type	Constraint	Description
bg_id	INT	PK, AUTO_INCREMENT	Unique blood group ID
group_name	VARCHAR(5)	UNIQUE NOT NULL	Blood group label (A+, O-, etc.)
description	TEXT	NULLABLE	Optional description or notes

5.2.4 contact_messages Table

Column	Data Type	Constraint	Description
msg_id	INT	PK, AUTO_INCREMENT	Unique message ID
sender_name	VARCHAR(100)	NOT NULL	Name of person who sent the message
sender_email	VARCHAR(150)	NOT NULL	Email address of the sender
subject	VARCHAR(255)	NULLABLE	Subject line of the message
message	TEXT	NOT NULL	Full message body
is_read	TINYINT(1)	DEFAULT 0	0 = unread, 1 = read by admin

Column	Data Type	Constraint	Description
sent_at	TIMESTAMP	DEFAULT NOW()	Message submission timestamp

5.3 Sample MySQL CREATE Statements

```

CREATE DATABASE bbdms_db;
USE bbdms_db;

CREATE TABLE donors (
    donor_id      INT AUTO_INCREMENT PRIMARY KEY,
    full_name     VARCHAR(100) NOT NULL,
    blood_group   VARCHAR(5) NOT NULL,
    age           INT NOT NULL,
    gender        ENUM('Male', 'Female', 'Other') NOT NULL,
    city          VARCHAR(100) NOT NULL,
    phone         VARCHAR(20) NOT NULL,
    email         VARCHAR(150),
    is_available  TINYINT(1) DEFAULT 1,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE users (
    user_id       INT AUTO_INCREMENT PRIMARY KEY,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(150) UNIQUE NOT NULL,
    password      VARCHAR(255) NOT NULL,
    role          ENUM('user', 'admin') DEFAULT 'user',
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

6. Backend / PHP Design

6.1 PHP File Structure and Routing

VES uses a page-based PHP routing model. Each feature corresponds to a specific PHP file. Form submissions use POST method; data retrieval uses GET. All database interactions use PHP MySQLi prepared statements to prevent SQL injection.

PHP File	Method	Functionality
index.php	GET	Home page — displays hero section and platform info
register.php	GET/POST	Donor registration form and insert to DB

PHP File	Method	Functionality
login.php	GET/POST	User/admin login; validates credentials; starts session
logout.php	GET	Destroys PHP session and redirects to login
donor_list.php	GET	Displays paginated list of all available donors
donor_detail.php	GET	Shows full profile of a single donor by donor_id
search.php	GET/POST	Search donors by blood group and/or city
contact.php	GET/POST	Public contact form; inserts message to DB
about.php	GET	About Us page with platform information
admin/index.php	GET	Admin dashboard with statistics
admin/manage_donors.php	GET/POST	Admin — view, edit, delete all donor records
admin/add_donor.php	GET/POST	Admin — manually add a new donor
admin/edit_donor.php	GET/POST	Admin — edit existing donor by donor_id
admin/delete_donor.php	GET	Admin — delete donor by donor_id
admin/messages.php	GET	Admin — view all contact messages
admin/stats.php	GET	Admin — total donors, blood group counts, city breakdown

Table 6.1 — PHP File Routing Table

6.2 Session and Authentication Flow

Step 1: User navigates to login.php and submits email + password

Step 2: PHP fetches user record from MySQL WHERE email = ? (prepared statement)

Step 3: PHP verifies submitted password against stored hash using password_verify()

Step 4: On success: \$_SESSION['user_id'] and \$_SESSION['role'] are set

Step 5: All protected pages begin with session_start() and check \$_SESSION['role']

Step 6: Admin pages additionally check if role === 'admin'; else redirect to home

Step 7: logout.php calls session_destroy() and redirects to index.php

6.3 Database Connection

```
// config/db.php
<?php
$host      = 'localhost';
$dbname    = 'bbdms_db';
$username  = 'root';
$password  = '';
$conn = new mysqli($host, $username, $password, $dbname);
if ($conn->connect_error) {
    die('Connection failed: ' . $conn->connect_error);
}
?>
```

7. User Interface Design

7.1 Design Principles

Clean, minimal design with a red and white color theme (blood donation branding)

Fully responsive using Bootstrap grid system

Navigation bar visible on all pages with key links

Forms include client-side JavaScript validation before submission

Admin panel has a separate layout with sidebar navigation

7.2 Page Inventory

Page Name	PHP File	Access	Key Elements
Home	index.php	Public	Hero banner, platform intro, quick search bar, statistics
About Us	about.php	Public	Platform mission, blood donation facts, team info
Donor List	donor_list.php	Public	Paginated table of donors with blood group and city
Search Donor	search.php	Public	Search form (blood group dropdown + city input), results table
Donor Detail	donor_detail.php	Public	Full donor profile card with contact details
Register	register.php	Public	Donor registration form with all required fields
Login	login.php	Public	Email + password form, role-based redirect on success
Contact Us	contact.php	Public	Name, email, message form; success confirmation
Admin Dashboard	admin/index.php	Admin Only	Stats cards: total donors, blood groups, messages, users
Manage Donors	admin/manage_donors.php	Admin Only	Data table with edit/delete actions per donor
Add Donor	admin/add_donor.php	Admin Only	Form for admin to manually add donor
Edit Donor	admin/edit_donor.php	Admin Only	Pre-filled edit form loaded by donor_id

Page Name	PHP File	Access	Key Elements
Messages	admin/messages.php	Admin Only	Inbox table of all contact form submissions

Table 7.1 — VES Page Inventory

7.3 Navigation Structure

Public Navbar (visible to all): Home | About Us | Contact Us | Donor List | Search Donor | Login
Admin Panel Sidebar: Dashboard | Manage Donors | Add Donor | Messages | Statistics | Logout

8. Security Design

8.1 Authentication and Session Security

- Passwords are hashed using PHP password_hash() with PASSWORD_BCRYPT before storage
- Login verification uses password_verify() — raw passwords are never compared directly
- PHP sessions are used for maintaining login state — session_start() on every protected page
- Sessions expire on browser close or after configurable inactivity period
- Admin pages include role check: if(\$_SESSION['role'] !== 'admin') redirect to index.php

8.2 Security Controls Table

Threat	Control Applied	Implementation Detail
SQL Injection	Prepared Statements	All MySQL queries use MySQLi prepared statements with bound parameters — no raw user input in SQL strings
XSS Attacks	Output Encoding	All user data rendered in HTML is wrapped with htmlspecialchars() to prevent script injection

Threat	Control Applied	Implementation Detail
Password Theft	bcrypt Hashing	PHP password_hash(PASSWORD_BCRYPT) with salt — plain passwords never stored in database
Unauthorized Access	Session Role Check	Every admin page validates \$_SESSION['role'] === 'admin'; redirect to login if check fails
CSRF	Form Token Validation	Hidden CSRF tokens generated per session and validated on POST submissions
Direct URL Access	Auth Middleware	Protected PHP pages check session at top of file; unauthenticated requests are redirected
Data Exposure	Sensitive Field Protection	Passwords never returned in SELECT queries for display; only hashed values stored

Table 8.1 — VES Security Controls

9. Use Case and Sequence Design

This section describes the step-by-step flow of the three most critical use cases in VES, showing the interaction between the user, PHP backend, and MySQL database.

9.1 Sequence: Donor Registration

Step	Actor	User Action / Browser	PHP (Server)	MySQL (Database)
1	Visitor	Opens register.php page	Renders registration form	—
2	Visitor	Fills form fields and clicks Submit	Receives POST data	—
3	System	—	Validates all inputs (empty check, age range, phone format)	—

Step	Actor	User Action / Browser	PHP (Server)	MySQL (Database)
4	System	—	Prepares INSERT statement with bound parameters	—
5	System	—	Executes INSERT query	INSERT INTO donors (...)
6	System	Success message shown	Returns success confirmation	—

Table 9.1 — Donor Registration Sequence

9.2 Sequence: Search Donor

Step	Actor	User Action / Browser	PHP (Server)	MySQL (Database)
1	Visitor	Opens search.php; selects blood group and/or city	Renders search form	—
2	Visitor	Clicks Search button	Receives GET/POST parameters	—
3	System	—	Sanitizes inputs; builds prepared query with WHERE conditions	—
4	System	—	Executes SELECT with bound parameters	SELECT * FROM donors WHERE blood_group=? AND city LIKE ?
5	System	—	Fetches result rows	Returns matching donor records

Step	Actor	User Action / Browser	PHP (Server)	MySQL (Database)
6	System	Results table rendered in browser	Passes data to HTML template	—

Table 9.2 — Search Donor Sequence

9.3 Sequence: Admin Deletes Donor

Step	Actor	User Action / Browser	PHP (Server)	MySQL (Database)
1	Admin	Logs in; navigates to manage_donors.php	Validates admin session	—
2	Admin	Clicks Delete button for a donor record	Receives donor_id via GET	—
3	System	—	Validates donor_id is a valid integer	—
4	System	—	Prepares DELETE statement	DELETE FROM donors WHERE donor_id = ?
5	System	—	Executes DELETE; checks affected rows	Removes donor record
6	System	Redirected with success message	Redirects with \$_GET['msg']	—

Table 9.3 — Admin Delete Donor Sequence

10. Non-Functional Design Decisions

Req. ID	Category	Requirement	Design Decision
NFR001	Performance	Fast page load	MySQL indexes on blood_group and city columns for fast search queries; PHP output buffering enabled
NFR002	Reliability	No data loss	MySQL ACID-compliant transactions; daily database backup via mysqldump; graceful error handling with PHP try-catch
NFR003	Security	Secure auth and data	bcrypt password hashing; prepared statements; session-based role control; htmlspecialchars() on all output
NFR004	Usability	Easy to use	Bootstrap responsive design; search available without login; form labels and error messages clearly displayed
NFR005	Scalability	Handle growing data	MySQL supports millions of records; pagination implemented on donor list to limit per-page data load
NFR006	Maintainability	Clean code structure	MVC-style folder organization; PHP functions separated into include files; inline comments on all major code blocks
NFR007	Compatibility	Cross-browser support	Bootstrap 4/5 provides cross-browser CSS compatibility; tested on Chrome, Firefox, and Edge
NFR008	Data Integrity	No partial writes	MySQL constraints (NOT NULL, UNIQUE) enforced at DB level; PHP validation before every INSERT/UPDATE

Table 10.1 — Non-Functional Design Decisions

11. Project Folder Structure

bbdms/	<- Root project folder (served from httdocs/)
— index.php	<- Home page
— about.php	<- About Us page
— contact.php	<- Contact form

```

— login.php          <- Login page
— logout.php         <- Session destroy and redirect
— register.php       <- Donor registration form
— donor_list.php     <- Public donor listing (paginated)
— donor_detail.php   <- Single donor profile view
— search.php         <- Donor search page

— admin/             <- Admin panel (role-protected)
  — index.php        <- Admin dashboard with stats
  — manage_donors.php <- View/edit/delete all donors
  — add_donor.php    <- Manually add a donor
  — edit_donor.php   <- Edit donor by ID
  — delete_donor.php <- Delete donor by ID
  — messages.php     <- View contact messages

— config/db.php      <- MySQL database connection
— includes/header.php <- Common HTML header + navbar
— includes/footer.php <- Common HTML footer
— includes/auth_check.php <- Session authentication guard
— assets/css/style.css <- Custom styles
— assets/js/validation.js <- Client-side form validation
— sql/bbdsms_db.sql  <- Full database schema + seed data

```

12. Design Rationale

Design Choice	Alternative Considered	Rationale
PHP (Backend)	Node.js / Python Django	PHP is widely taught in Pakistani university curriculum; runs natively on XAMPP/Apache without additional setup; familiar to academic evaluators
MySQL (Database)	Firebase / MongoDB	MySQL is relational and ideal for structured donor data with fixed schema; well-supported by PHP via MySQLi; standard in academic FYP projects
Bootstrap (Frontend)	Tailwind CSS / Custom CSS	Bootstrap provides ready-made responsive components; reduces development time; standard for PHP-based web projects
PHP Sessions (Auth)	JWT Tokens	PHP sessions are simpler and native to PHP; JWT requires additional libraries; sessions are sufficient for a single-server academic project
Prepared Statements	Raw SQL Queries	Prepared statements prevent SQL injection which is the most critical web security risk; no performance cost for this project scale

Design Choice	Alternative Considered	Rationale
Page-Based Routing	MVC Framework (Laravel)	Simple file-based routing is easier to understand and present in a viva; no framework dependency; transparent code flow for evaluators
bcrypt Password Hashing	MD5 / SHA1	bcrypt is the industry standard for password hashing with built-in salting; MD5/SHA1 are cryptographically broken and must never be used for passwords

Table 12.1 — VES Design Rationale

13. References

1. Muhammad Adil Nadeem, "Software Requirements Specifications — Visually Essential Social (VES)," Islamia University of Bahawalpur, 2025.
2. IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, Jun. 1998.
3. PHP Group, "PHP Manual," [Online]. Available: <https://www.php.net/manual/en/>. [Accessed: 2025].
4. Oracle Corporation, "MySQL 8.0 Reference Manual," [Online]. Available: <https://dev.mysql.com/doc/>. [Accessed: 2025].
5. Bootstrap Team, "Bootstrap Documentation," [Online]. Available: <https://getbootstrap.com/docs>. [Accessed: 2025].
6. OWASP Foundation, "OWASP Top Ten Web Application Security Risks," [Online]. Available: <https://owasp.org/Top10>. [Accessed: 2025].
7. W3Schools, "PHP Tutorial," [Online]. Available: <https://www.w3schools.com/php/>. [Accessed: 2025].
8. GitHub Inc., "GitHub Documentation," [Online]. Available: <https://docs.github.com>. [Accessed: 2025].
9. Apache Friends, "XAMPP Documentation," [Online]. Available: <https://www.apachefriends.org>. [Accessed: 2025].
10. World Health Organization, "Blood Safety and Availability," WHO Fact Sheet, June 2023.